System Architecture Overview

This model is a **multi-stage machine learning system** that predicts profitable trades around earnings announcements for AAPL, NVDA, and GOOGL. It combines traditional financial modelling with modern ML techniques and sophisticated validation methods.

Stage 1: Data Foundation

Real Data Collection:

- **Earnings Data:** EPS estimates, reported EPS, surprise percentages from Yahoo Finance
- **Price Data:** Daily returns from T-10 to T+20 around each earnings event (31- day window)
- **Macro Data:** Federal funds rate, Core CPI, unemployment from FRED
- **Sentiment Data:** Alpha Vantage news sentiment + tweet analysis

Data Scope:

- **Time Period:** 2015-2025 (10 years)
- **Real Events:** ~42 per stock (126 total)
- **Quality Controls:** Removes extreme returns (>50% daily), validates data completeness

Stage 2: Feature Engineering

Core Sub-scores (normalised to -1 to +1 scale):

- Post-earnings: Actual surprise vs estimates
- Pre-earnings: Current estimate vs historical rolling average

EPS Subscore:

◦ Z-score normalised to prevent outliers

Macro Subscore:
◦ Combines EFFR (-), Core CPI (-), Unemployment (-)

◦ Negative signs mean lower rates/inflation = positive score

◦ Captures economic regime (expansion/contraction)

Volatility Subscore:
◦ Pre-earnings volatility (T-10 to T-2)

◦ Inverted: lower volatility = higher score

◦ Indicates market stability before announcement

Sentiment Subscore:

◦ 60% Alpha Vantage news sentiment

◦ 40% tweet sentiment analysis

◦ Enhanced features for NVDA (AI hype, data center sentiment)

Revenue Subscore:

◦ Proxy from EPS surprise (Revenue ≈ EPS surprise × 0.7)

◦ Enhanced for NVDA with segment-specific growth rates

Stage 3: Composite Score Generation

Stock-Specific Weights:

AAPL: EPS(30%) + Macro(20%) + Vol(15%) + Sentiment(30%) + Revenue(5%) NVDA: EPS(30%) + Macro(15%) + Vol(10%) + Sentiment(40%) + Revenue(5%) GOOGL: EPS(25%) + Macro(25%) + Vol(15%) + Sentiment(30%) + Revenue(5%)

Two Composite Scores:

- **Pre-earnings:** Uses EPS estimates (no forward bias)
- **Post-earnings:** Uses actual EPS results (for backtesting)

Normalization: Clips to [-0.3, +0.3] to prevent extreme signals

Stage 4: Synthetic Data Generation

Problem: Only 42 real events per stock insufficient for ML training

Solution: Generate 1,260 synthetic events per stock using 4 methods:

1. **Bootstrap (35%):** Resample similar historical events with noise
2. **Volatility-based (25%):** Scale returns based on volatility regime
3. **Macro-regime (20%):** Adjust returns based on economic conditions
4. **Rolling window (20%):** Use historical averages with time-dependent noise

Quality Assurance: Maintains realistic correlations and distributions

Stage 5: Adaptive Threshold Optimisation

Multi-Level Threshold System

Level 1 - Base Optimisation:

- Tests 104 combinations (26 base thresholds × 4 multipliers)
- Uses cross-validation or time-series CV
- Optimises for combined hit rate and returns

Level 2 - Market Condition Adjustments:

Applied to each event individually
Macro Adjustment: Bullish(-0.05), Bearish(+0.05), Neutral(0) Sentiment Adjustment: Strong(±0.03), Weak(0)
Final Threshold = Base Threshold + Adjustments

python

Applied to each event individually
Macro Adjustment: Bullish(-0.05), Bearish(+0.05), Neutral(0) Sentiment Adjustment: Strong(±0.03), Weak(0)
Final Threshold = Base Threshold + Adjustments

Level 3 - Stock-Specific Refinements:

- NVDA: Enhanced short bias for "sell the news" pattern
- AAPL: Balanced approach with quality focus
- GOOGL: Macro-sensitive with quick reactions

Stage 6: Signal Generation Logic

Three Decision Zones:

LONG Signal: Composite Score ≥ Adaptive Long Threshold

SHORT Signal: Composite Score ≤ Adaptive Short Threshold NEUTRAL: Between thresholds (no trade)

Dynamic Thresholds: Each event gets individually adjusted thresholds based on market conditions at that time.

Stage 7: ML Model Integration Model Pipeline:

- **Random Forest Classifier:** Binary profit/loss prediction
- **Random Forest Regressor:** Return magnitude prediction

- **Logistic Regression:** Linear classification
- **Decision Trees:** Interpretable models

Model Selection: If ML models available, uses predictions; otherwise falls back to composite scores.

Stage 8: Strategy Execution & Performance Calculation Return Calculation:

- **Entry:** T-5 (5 days before earnings)
- **Exit:** T+5 (5 days after earnings)
- **Long Returns:** Direct strategy returns
- **Short Returns:** Inverted strategy returns (profit when stock drops)

Performance Metrics:

- **Hit Rate:** % of profitable trades
- **Average Return:** Mean return per trade
- **Total Return:** Compounded returns
- **Sharpe Ratio:** Risk-adjusted returns (with 4.33% risk-free rate)

Stage 9: Enhanced Validation Methods

Time-Series Cross-Validation:

- Uses each stock's own historical data
- 3-fold chronological splits
- Prevents future data leakage
- Captures stock-specific patterns

Traditional Cross-Validation:

- Uses other stocks' data for optimisation
- Tests generalisability across companies
- Prevents overfitting to specific stock

Key Innovations

1. Adaptive Market Regime Recognition: Thresholds change based on macro and sentiment conditions

2. Stock-Specific Pattern Recognition:

- NVDA's "sell the news" pattern
- AAPL's stability premium
- GOOGL's macro sensitivity

3. Synthetic Data Augmentation: Expands limited dataset while maintaining realistic properties
4. Multi-Model Ensemble: Combines ML predictions with traditional composite scoring

Critical Assessment

Strengths:

- Sophisticated multi-factor approach
- Proper temporal validation
- Stock-specific optimisations
- Robust data quality controls

Potential Weaknesses:

- **Limited Sample Size:** 42 real events may not support this level of complexity
- **Overfitting Risk:** Testing thousands of parameters on small dataset
- **Forward Bias:** Post-earnings EPS usage in backtests
- **Transaction Cost Reality:** Real-world costs could eliminate edge

Model Assumptions:

1. Historical patterns persist
2. 31-day windows capture all relevant price action
3. Synthetic data accurately reflects real market dynamics
4. Sentiment proxies capture actual market sentiment

How Predictions Are Made

1. Data Input: Collect EPS estimate, macro data, sentiment, pre-earnings volatility

2. Sub-score Calculation: Generate 5 normalised subscores

3. Composite Score: Apply stock-specific weights

4. Threshold Calculation: Adjust base thresholds for current market conditions

5. Signal Generation: Compare composite score to adaptive thresholds

6. Position Sizing: Apply Kelly criterion

7. Risk Assessment: Consider VIX regime

The system learns that certain combinations of earnings surprise expectations, market conditions, sentiment, and volatility predict profitable trading opportunities around earnings announcements, with different patterns for each stock.

The model represents a sophisticated approach to earnings trading, but the complexity relative to available data raises questions about generalisability to live trading conditions.

Z-Score

What it is: A statistical measure that tells you how many standard deviations a data

point is from the average.

Formula: $Z = (X - \mu) / \sigma$

- X = individual value
- μ = mean of all values
- σ = standard deviation

Example:

python

 # EPS surprises for AAPL: [2%, 5%, -1%, 8%, 3%] # Mean = 3.4%, Standard deviation = 3.2%

 # For an 8% surprise:

Z = (8% - 3.4%) / 3.2% = 1.44
 # This means 8% surprise is 1.44 standard deviations above average

Enhanced tools

- **Normalization:** Makes different stocks comparable (AAPL's 2% surprise ≠ NVDA's 2% surprise in impact)
- **Outlier Control:** Extreme values get capped at ±3 standard deviations
- **Statistical Significance:** Shows which earnings surprises are truly unusual

EFFR (Effective Federal Funds Rate)

What it is: The actual interest rate banks charge each other for overnight loans.

The current federal funds rate is around 4.50%, down from 5.33% in 2023 due to recent Fed cuts in late 2024 TradingViewYCharts.

The way we use EFFR:

- **Economic Regime Indicator:** High EFFR = tight monetary policy, Low EFFR = loose policy
- **Negative Weight:** Your system gives EFFR a negative weight because lower rates are generally bullish for stocks
- **Macro Subscore Component:** Combined with CPI and unemployment to gauge overall economic conditions

Example:

python

 # Current EFFR = 4.50%, Historical average = 3.0%
 # Higher than average = tighter policy = negative for stocks # Z-score = (4.50 - 3.0) / 1.5 = +1.0
 # Macro subscore gets -1.0 contribution (negative weight)

Revenue Subscore Proxy

The Problem: the system doesn't have actual revenue surprise data for all earnings events.
The Solution: Use EPS surprise as a proxy since revenue and EPS are correlated.

The Logic:

- Companies that beat EPS estimates often also beat revenue estimates
- But revenue is typically less volatile than EPS
- So revenue surprise ≈ 70% of EPS surprise magnitude

Example:

Python

 # AAPL reports +15% EPS surprise
 # Revenue surprise proxy = 15% × 0.7 = +10.5%

 # This assumes:
 # - Strong EPS performance indicates strong revenue performance

 # - But revenue growth is typically more stable than earnings growth # - The 0.7 factor dampens the EPS volatility

Enhanced Version for NVDA: the system creates a more sophisticated revenue proxy for NVDA:

python

 # NVDA gets segment-specific treatment:

enhanced_revenue = (data_center_growth * 0.5 + gaming_growth * 0.3 + professional_growth * 0.2

)

 # 50% weight (biggest segment) # 30% weight

 # 20% weight

Clipping to [-0.3, +0.3]

What Clipping Means: Setting maximum and minimum boundaries on values:

- Prevents any single factor from dominating the composite score
- Ensures balanced signal distribution
- Stops extreme outliers from creating unrealistic signals

Without Clipping (Problem):

Python

 # Extreme example:
eps_score = 2.5 *# Massive earnings beat* macro_score = -0.1 *# Slightly bearish macro*
sentiment_score = 0.2 *# Slightly positive*

composite = 2.5 * 0.3 + (-0.1) * 0.2 + 0.2 * 0.3 = 0.75 + (-0.02) + 0.06 = 0.79 *# This would ALWAYS trigger a long signal, regardless of other factors*

With Clipping (Solution): python

 # Same example with clipping:
eps_score = min(max(2.5, -0.3), 0.3) = 0.3 *# Clipped to max* macro_score = -0.1 *# Within bounds*
sentiment_score = 0.2 *# Within bounds*

composite = 0.3 * 0.3 + (-0.1) * 0.2 + 0.2 * 0.3 = 0.09 + (-0.02) + 0.06 = 0.13 *# Now other factors can still influence the decision*

Stock-Specific Clipping for NVDA: Your system allows NVDA slightly higher positive signals:

python

 # NVDA gets [-0.25, +0.35] range instead of [-0.3, +0.3]
 # This captures NVDA's potential for explosive positive reactions

Model Pipeline

What a Pipeline Is: A sequence of data processing steps that transform raw inputs

into final predictions.

Model Pipeline Steps: the Model's Pipeline Flow

Document
Earnings Model Pipeline ## Step 1: Raw Data Input ``` ├── EPS Estimate: $1.25 ├── Reported EPS: $1.42 ├── EFFR: 4.50% ├── Core CPI: 327.6 ├── Unemployment: 4.1% ├── Pre-earnings volatility: 0.025 ├── Sentiment score: 0.15 └── Stock symbol: AA

The pipeline is essentially the system's "assembly line" - it takes raw earnings data through a standardised series of transformations to produce a final trading decision.

Key Pipeline Concepts:

1. **Sequential Processing:** Each step depends on the previous step's output
2. **Standardization:** Same process applied to every stock and every earnings

event

3. **Multiple Models:** the pipeline can use different ML models (Random Forest,

Decision Trees, Logistic Regression) and choose the best one

4. **Fallback Logic:** If ML models fail, falls back to composite scores
5. **Quality Gates:** Each step validates inputs before processing

Critical Assessment of These Components

Strengths:

- **Z-score normalization** makes different stocks comparable
- **EFFR integration** captures monetary policy impact

- **Revenue proxy** addresses missing data creatively
- **Clipping** prevents extreme outliers from dominating
- **Pipeline structure** ensures reproducible results

Potential Issues:

- **Revenue proxy assumption (**70% correlation) may not hold across all companies/sectors
- **Clipping boundaries** (±0.3) are somewhat arbitrary - why not ±0.25 or ±0.35?
- **Z-score stability** - with only 42 events per stock, mean/std estimates are unstable
- **Pipeline complexity** makes it hard to identify which component is driving performance
- **EFFR lag** - interest rate impacts may not be immediate around earnings

Recommendations:

1. **Validate revenue proxy** - check actual correlation between EPS and revenue surprises
2. **Test clipping sensitivity** - how much do results change with different boundaries?
3. **Rolling z-scores** - use expanding windows rather than full-sample statistics
4. **Component analysis** - test pipeline with individual components removed
5. **Simplification** - consider if a simpler model performs nearly as well

The pipeline represents sophisticated financial engineering, but the complexity relative to your limited data (42 events per stock) raises concerns about whether you're extracting signal or fitting noise.